DESHMUKH TECHNOLOGIES · DTTP

COMPLETE GUIDE · V1.0

Trainee Management System

The first real project for DTTP trainees. Build it as a genuine internal business application, not as a tutorial. This single PDF holds the full product: ten modules, MySQL, APIs, Sprint 1 setup, sprints 1–10, and the first user story.

> Read Parts I–III once. Build only Part IV (sections 18–19) now. Later modules are the map. Do not skip to attendance or Docker.

01 Project goal

Build a web application to manage the complete DTTP trainee lifecycle.

```
Registration → Batch Assignment → Training → Attendance
→ Tasks & Assignments → Evaluation → Feedback → Completion
```

02 Technology stack

BACKEND

Python · FastAPI · SQLAlchemy · MySQL · JWT · Pydantic · Pytest

FRONTEND

React · TypeScript · HTML/CSS · REST API

TOOLS

VS Code · Git · GitHub · Postman · MySQL Workbench · Docker · GitHub Actions

03 User roles

ADMIN

Complete system.

- Trainees
- Mentors
- Batches
- Projects
- Users
- Reports

MENTOR

Assigned trainees.

- Attendance
- Tasks
- Assignments
- Reviews
- Feedback
- Performance

TRAINEE

Own activities.

- Profile
- Attendance
- Tasks
- Assignments
- Submissions
- Feedback
- Progress

04 Module 1 — Authentication

Login · Logout · JWT · Role-based access · Password management. Roles: ADMIN MENTOR TRAINEE

```
POST /api/auth/login
{ "email": "admin@deshmukh.local", "password": "use-a-local-password" }
```

Result	HTTP
Success + token	200
Wrong email or password	401
Protected route, no token	401
Wrong role	403

Hash passwords. Never store plaintext. Never commit secrets.

05 Module 2 — Trainee management

Admin can add, view, edit, search, filter, activate/deactivate, assign batch, and assign mentor.

Field	Rule
Trainee ID	Server-assigned
Name	Required
Email	Required, unique, lowercase
Mobile / address / education	Optional
Joining date	Required
Batch / mentor	Optional until assigned
Status	ACTIVE or INACTIVE. New rows are ACTIVE

`DELETE /api/trainees/{id}` deactivates the row. Do not run `DELETE FROM trainees`. Search: `GET /api/trainees?q=rohan&status=ACTIVE`

06 Module 3 — Batch management

Admin creates batches such as `DTTP-2026-01`, `DTTP-2026-02`, `DTTP-2026-03`.

Field	Rule
Batch name	Required, unique
Start date	Optional until the batch is ACTIVE
End date	Optional; must not be before start date
Mentor	Optional until assigned
Capacity	Optional positive integer
Status	PLANNED / ACTIVE / CLOSED

A trainee belongs to at most one active batch. A second active assign is **409**.

07 Module 4 — Mentor management

Admin can add, update, assign, view assigned trainees, and activate/deactivate a mentor. Name required. Email required, unique, lowercase. Status ACTIVE or INACTIVE.

08 Module 5 — Attendance

Mentor marks Present, Absent, Late, or Leave.

Date	Trainee	Status
19-Aug	Rahul	Present
19-Aug	Amit	Absent
19-Aug	Sana	Present

Same trainee + same date twice: **409**. Reports: daily, monthly, individual, attendance percentage.

09 Module 6 — Task management

Fields: Title, Description, Priority, Deadline, Assigned To. Status flow: TODO → IN_PROGRESS → SUBMITTED → UNDER_REVIEW → COMPLETED. Illegal jumps **409**.

TODO IN_PROGRESS SUBMITTED UNDER_REVIEW COMPLETED

10 Module 7 — Assignment and submission

```
Mentor → Create Assignment → Trainee → Complete Work → Submit → Mentor Review → Feedback
```

Submission: description, GitHub repository, file/document, submission date, comments. Unique pair (`trainee_id, task_id`). A second assign of the same task is **409**.

11 Module 8 — Evaluation

Skill	Score
Python	8/10
FastAPI	7/10
MySQL	8/10
React	7/10
Git	9/10
Problem Solving	8/10
Communication	8/10

12 Module 9 — Feedback

Mentor provides technical, task, and project feedback, plus strengths, areas for improvement, and recommendations. Trainee views feedback from the dashboard.

13 Module 10 — Dashboard

ADMIN

- Total / active trainees
- Total batches and mentors
- Today's attendance
- Pending tasks and reviews

MENTOR

- My trainees
- Today's attendance
- Pending tasks
- Pending submissions
- Performance

TRAINEE

- My profile
- Attendance %
- Pending / completed tasks
- Assignments
- Performance and feedback

14 Database design

Tables: `users` · `roles` · `trainees` · `mentors` · `batches` · `attendance` · `tasks` · `assignments` · `submissions` · `evaluations` · `feedback`

```
User → Trainee | Mentor
Batch → Trainees
Mentor → Trainees
Trainee → Attendance, Tasks, Assignments, Submissions, Evaluations, Feedback
```

MVP tables (Sprint 1–2). Database name: `dttp_tms`. Character set `utf8mb4`.

```
CREATE DATABASE IF NOT EXISTS dttp_tms
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  email VARCHAR(190) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
);

CREATE TABLE batches (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(80) NOT NULL UNIQUE,
  start_date DATE NULL,
  end_date DATE NULL,
  capacity INT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'PLANNED'
);

CREATE TABLE trainees (
  id INT PRIMARY KEY AUTO_INCREMENT,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(190) NOT NULL UNIQUE,
  mobile VARCHAR(20) NULL,
  address VARCHAR(255) NULL,
  education VARCHAR(120) NULL,
  joining_date DATE NOT NULL,
  batch_id INT NULL,
  mentor_id INT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_trainee_batch FOREIGN KEY (batch_id) REFERENCES batches(id)
);
```

15 Backend architecture

```
FastAPI → Routers / Schemas / Services / Models / Database / Auth / Exceptions
                ↓
           SQLAlchemy → MySQL
```

```
backend/
├── app/
│   ├── main.py, config/, database/, models/
│   ├── schemas/, routers/, services/, auth/, exceptions/
├── tests/
├── requirements.txt
├── .env.example      ← never commit secrets
└── README.md
```

Router does not write raw SQL. Service does not return HTTP objects. Model is the table. Schema is the JSON. Never commit `.env`.

16 Frontend architecture

React: Pages · Components · Layouts · Services · Hooks · Routes · Types · Utils

All screens: Login, Dashboard, Trainees, Trainee Details, Batches, Mentors, Attendance, Tasks, Assignments, Submissions, Evaluation, Feedback, Profile.

MVP-1 screens only: Login, Admin Dashboard, Trainees (list / add / edit), Batches (list / add / edit).

17 Error shape

```
{ "error": "VALIDATION_FAILED", "message": "Email is required", "field": "email" }
```

error	HTTP	When
VALIDATION_FAILED	400	Missing or invalid field
UNAUTHORIZED	401	Bad login or missing token
FORBIDDEN	403	Wrong role
NOT_FOUND	404	Unknown id
CONFLICT	409	Duplicate email, duplicate attendance, illegal status jump

Empty list is **200 []**, not 404. Duplicate email is **409**.

18 First MVP — build only this

Do not build all modules at once. Do **not** build attendance, tasks, evaluation, feedback, Docker, or CI/CD yet.

```
Login → Admin Dashboard → Trainee Management → Batch Management
```

Method	Path	Success	Notes
POST	/api/auth/login	200	Token in body
POST	/api/trainees	201	Duplicate email 409
GET	/api/trainees	200	Empty list is []
GET	/api/trainees/{id}	200	Unknown id 404
PUT	/api/trainees/{id}	200	Unknown id 404
DELETE	/api/trainees/{id}	200	Sets INACTIVE
POST	/api/batches	201	Duplicate name 409
GET	/api/batches	200	Empty list is []
GET	/api/batches/{id}	200	Unknown id 404
PUT	/api/batches/{id}	200	Unknown id 404
GET	/api/health	200	{ "status": "ok" }

`POST /api/trainees` example. Success **201**.

```
{
  "full_name": "Rohan Deshmukh",
  "email": "rohan.deshmukh@example.com",
  "mobile": "9876543210",
  "joining_date": "2026-08-19",
  "batch_id": 1
}
```

Sprints 1–4 finish this MVP. Stop after Sprint 4 until the mentor opens Sprint 5.

19 Sprint 1 — project setup

Do this before Trainee CRUD. Branch **`tms/sprint-1-setup`**. No feature APIs yet.

1. Create the GitHub repo (or use the one the mentor assigned). Clone with SSH.
2. Create branch **`tms/sprint-1-setup`**.
3. Create this layout:

```
tms/
├── backend/app/main.py
├── backend/requirements.txt
├── backend/.env.example
├── frontend/          Vite React + TypeScript
├── README.md
└── .gitignore
```

4. Create MySQL database **`dttp_tms`** and user **`tms_dev`** (local password, not in chat).
5. FastAPI **`GET /api/health`** returns **`{ "status": "ok" }`**.
6. React **`npm run dev`** loads in Chrome.
7. README: Python version, how to create the database, install, run backend, run frontend.
8. Commit, push, open a pull request.

```
MYSQL_HOST=localhost
MYSQL_PORT=3306
MYSQL_USER=tms_dev
MYSQL_PASSWORD=use-a-local-password
MYSQL_DATABASE=dttp_tms
```

Done when: GitHub repo exists, health is ok, MySQL **`dttp_tms`** exists, React opens in Chrome, README is usable, mentor reviewed the PR. Then Sprint 2 — not before.

20 Development sprints

Sprint	Work	Start
1	Repo, FastAPI, React, MySQL, README	**Now**
2	Trainee + batch models, CRUD APIs, Postman	After Sprint 1
3	React login, dashboard, trainee list, add/edit	After Sprint 2
4	JWT, roles, protected APIs	After Sprint 3
5	Batch polish, mentor, assignment	After MVP-1 Pass
6	Attendance + reports	After Sprint 5
7	Tasks, submit, review	After Sprint 6
8	Evaluation, feedback, progress	After Sprint 7
9	Unit, API, frontend tests	After Sprint 8
10	Actions → Tests → Build → Docker → Deploy	After Sprint 9

21 First real-time requirement

> As an Admin, I want to register DTTP trainees and assign them to a batch so that I can maintain a centralized trainee database.

- [ ] Admin can log in
- [ ] Admin can create a trainee
- [ ] Required fields are validated
- [ ] Email cannot be duplicated
- [ ] Admin can view trainees
- [ ] Admin can search trainees
- [ ] Admin can edit trainee details
- [ ] Admin can deactivate a trainee
- [ ] Admin can assign a batch
- [ ] Data is stored in MySQL
- [ ] APIs are tested in Postman
- [ ] Frontend works
- [ ] Code is committed through Git
- [ ] Pull request is reviewed by mentor

22 Trainee development workflow

```
Requirement → GitHub Issue → Feature Branch → Design → Code → Test
→ Commit → Push → Pull Request → Mentor Review → Fix → Approval → Merge
```

The first project should teach how real software teams work, not merely how to write Python.

> First milestone: build and deploy a working TMS MVP using Python + FastAPI + MySQL + React, with GitHub-based code review.